



`</>` **main.py**
Python

`</>` **game_engine.py**
Python

`</>` **balloon.py**
Python

`</>` **click_detection.py**
Python

`</>` **renderer.py**
Python

I'm working on a Pygame "Balloon Pop" game for a lab. I'll share the code

file by file. There is one deliberate bug and I need to add 3 features

step by step. Please don't change anything until I ask.

Rules for your

replies: give me the complete updated code for every file you change,

keep the folder structure (main.py, game/game_engine.py, game/balloon.py,

game/click_detection.py, game/renderer.py), keep the code simple and

object-oriented, and briefly explain what you changed and why.

Here are the files:

--- main.py ---

--- game/game_engine.py ---

--- game/balloon.py ---

--- game/click_detection.py ---

--- game/renderer.py ---

Reply only with a short summary of how the game currently works.

Show less ^

The game is a simple **Pygame Balloon Pop** game:

- `main.py` initializes Pygame, handles the game loop, mouse clicks, updating, and drawing. `main`
- `game_engine.py` manages balloons, periodically spawns them with random sizes/speeds, moves them downward, removes balloons that leave the screen, and maintains the score. `game_engine`
- `balloon.py` defines the Balloon object, including its position, radius, speed, movement, and boundary check. `balloon`
- `click_detection.py` determines whether a mouse click hits a balloon, but currently contains the deliberate click-detection bug. `click_detection`
- `renderer.py` handles all visual rendering: background, balloons, strings, score text, and banners. `renderer`

Currently, the game has **one balloon type, scoring, spawning, movement, and clicking**, but **no lives or timer yet**. `game_engine`

Sources



Task 1: Fix a bug. In `game/click_detection.py`, `check_pop` computes the SQUARED distance from the click to the balloon center but compares it against the plain (non-squared) radius. So clicks only register near the exact center. A click anywhere inside the balloon's visible circle should pop it. Fix this so it compares actual distance to actual radius (either compare squared distance with `radius**2`, or use `math.hypot`). Give me the full corrected file and explain the bug in 2 lines.

For Task 2, I'm changing `game/balloon.py` and `game/game_engine.py` . The click-detection fix from Task 1 remains unchanged.

game/balloon.py

Python

Run

```
"""
Balloon: falls from the top of the screen. The player must pop it
before it reaches the bottom.
"""

import pygame

class Balloon:
    def __init__(self, x, y, radius, speed, balloon_type="normal"):
        self.x = x
        self.y = y
        self.radius = radius
        self.speed = speed
        self.balloon_type = balloon_type

        if balloon_type == "bonus":
            self.color = (255, 215, 0)    # Gold/Yellow
            self.points = 30
        elif balloon_type == "penalty":
            self.color = (80, 20, 20)    # Dark Red
            self.points = -20
        else:
            self.color = (220, 90, 120)  # Normal
            self.points = 10

    def update(self):
        self.y += self.speed

    def is_past_bottom(self, height):
        return self.y - self.radius > height

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius),
            int(self.y - self.radius),
            self.radius * 2,
            self.radius * 2
        )
```

game/game_engine.py

Python

Run

```
"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.
"""
```

```
"""
```

```
import random
```

```
from game.balloon import Balloon
```

```
from game.click_detection import check_pop
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 45
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.balloons = []
```

```
        self.frames_until_spawn = 0
```

```
        self.score = 0
```

```
    def _spawn_balloon(self):
```

```
        radius = random.randint(16, 44)
```

```
        x = random.randint(radius + 10, WIDTH - radius - 10)
```

```
        speed = random.uniform(1.5, 3.0)
```

```
        balloon_type = random.choices(
            ["normal", "bonus", "penalty"],
            weights=[65, 20, 15],
            k=1
        )[0]
```

```
        self.balloons.append(
            Balloon(
                x=x,
                y=-radius,
                radius=radius,
                speed=speed,
                balloon_type=balloon_type
            )
        )
```

```
    def handle_click(self, pos):
```

```
        popped = check_pop(self.balloons, pos)
```

```
        if popped is not None:
```

```
            self.balloons.remove(popped)
```

```
            self.score += popped.points
```

```
            self.score = max(0, self.score)
```

```
    def update(self):
```

```
        self.frames_until_spawn -= 1
```

```
        if self.frames_until_spawn <= 0:
```

```
            self._spawn_balloon()
```

```

        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for b in self.balloons:
        b.update()

    self.balloons = [
        b for b in self.balloons
        if not b.is_past_bottom(HEIGHT)
    ]

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(surface, self.balloons)
    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

```

What changed

- Added **Normal (+10)**, **Bonus (+30)**, and **Penalty (-20)** balloon types with different colors.
- Balloons spawn with **65% / 20% / 15%** weights, and popping applies the balloon's points while keeping the score at **0 or above**.
- The Task 1 squared-distance click fix is preserved.



Task 3: Add a lives system. The player starts with 3 lives. When ANY balloon reaches the bottom of the screen without being popped, the player loses one life. Popping a balloon must NEVER cost a life, whatever its type. Display the remaining lives on screen (for example "Lives: 3" or heart icons). When lives reach 0, the game ends (for now, just stop the game and show "Game Over"). Make sure a balloon that is popped is removed immediately so it can't also count as a miss. Give me the full updated files.

For Task 3, only `game/game_engine.py` and `main.py` need to change. The balloon types and Task 1 click-detection fix remain intact.

```
"""
GameEngine: owns all balloons, spawns new ones, handles clicks,
and manages the player's lives.
"""

import random

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45

class GameEngine:
    def __init__(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0
        self.lives = 3
        self.game_over = False

    def _spawn_balloon(self):
        radius = random.randint(16, 44)
        x = random.randint(radius + 10, WIDTH - radius - 10)
        speed = random.uniform(1.5, 3.0)

        balloon_type = random.choices(
            ["normal", "bonus", "penalty"],
            weights=[65, 20, 15],
            k=1
        )[0]

        self.balloons.append(
            Balloon(
                x=x,
                y=-radius,
                radius=radius,
                speed=speed,
                balloon_type=balloon_type
            )
        )

    def handle_click(self, pos):
        if self.game_over:
            return
```

```

        popped = check_pop(self.balloons, pos)

        if popped is not None:
            # Remove immediately so it cannot later count as a miss.
            self.balloons.remove(popped)

            self.score += popped.points
            self.score = max(0, self.score)

def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_balloon()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    missed_balloons = []

    for balloon in self.balloons:
        balloon.update()

        if balloon.is_past_bottom(HEIGHT):
            missed_balloons.append(balloon)

    # Every balloon that reaches the bottom costs one life.
    for balloon in missed_balloons:
        self.balloons.remove(balloon)
        self.lives -= 1

    if self.lives <= 0:
        self.lives = 0
        self.game_over = True

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(surface, self.balloons)

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,

```

```

        f"Lives: {self.lives}",
        (10, 40)
    )

    if self.game_over:
        renderer.draw_banner(surface, font, "Game Over")

```

main.py

Python

Run

```

"""
Balloon Pop (Lab Starter)

Run with: python3 main.py

Click balloons to pop them before they reach the bottom.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Balloon Pop")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

```

```
if __name__ == "__main__":  
    main()
```

What changed

- Added 3 lives, with one life lost for every balloon that reaches the bottom.
- Popped balloons are removed immediately, so they **cannot also count as missed balloons**.
- Added Lives: X display and Game Over when lives reach 0.
- Once the game is over, balloons stop updating/spawning and clicks no longer affect the game.



Task 4: Add a timed round. Add a 30-second countdown displayed on screen. The round ends when the timer hits 0 OR lives hit 0, whichever comes first. When the round ends: stop spawning balloons, ignore clicks on balloons, show a clear "Game Over" screen with the FINAL SCORE, and let the player press R (or click a Restart button) to start a new round. The restart must reset the score, lives, timer and clear all existing balloons. Use pygame time (not time.sleep). Give me the full updated files.

For Task 4, I'm updating `main.py`, `game/game_engine.py`, and `game/renderer.py`. The balloon types, lives system, and Task 1 click fix are preserved.

`main.py`

Python

Run

```
"""
```

```
Balloon Pop
```

```
Run with: python3 main.py
```

```
Click balloons to pop them before they reach the bottom.
```

```
Press R or click Restart after the round ends.
```

```
"""
```

```
import pygame
```

```
from game.game_engine import GameEngine
```

```

from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Balloon Pop")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)

            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    engine.restart()

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()

```

game/game_engine.py

Python

Run

```

"""
GameEngine: owns all balloons, spawns new ones, handles clicks,
and manages the player's lives and round timer.
"""

import random
import pygame

from game.balloon import Balloon
from game.click_detection import check_pop

```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 45
```

```
ROUND_DURATION_SECONDS = 30
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.balloons = []
```

```
        self.frames_until_spawn = 0
```

```
        self.score = 0
```

```
        self.lives = 3
```

```
        self.game_over = False
```

```
        self.start_time = pygame.time.get_ticks()
```

```
        self.time_left = ROUND_DURATION_SECONDS
```

```
    def restart(self):
```

```
        self.balloons.clear()
```

```
        self.frames_until_spawn = 0
```

```
        self.score = 0
```

```
        self.lives = 3
```

```
        self.game_over = False
```

```
        self.start_time = pygame.time.get_ticks()
```

```
        self.time_left = ROUND_DURATION_SECONDS
```

```
    def _spawn_balloon(self):
```

```
        radius = random.randint(16, 44)
```

```
        x = random.randint(radius + 10, WIDTH - radius - 10)
```

```
        speed = random.uniform(1.5, 3.0)
```

```
        balloon_type = random.choices(
            ["normal", "bonus", "penalty"],
            weights=[65, 20, 15],
            k=1
        )[0]
```

```
        self.balloons.append(
            Balloon(
                x=x,
                y=-radius,
                radius=radius,
                speed=speed,
                balloon_type=balloon_type
            )
        )
```

```
    def handle_click(self, pos):
```

```
        if self.game_over:
```

```
            return
```

```
        popped = check_pop(self.balloons, pos)
```

```

        if popped is not None:
            # Remove immediately so it cannot count as a miss.
            self.balloons.remove(popped)

            self.score += popped.points
            self.score = max(0, self.score)

def update(self):
    if self.game_over:
        return

    elapsed_seconds = (
        pygame.time.get_ticks() - self.start_time
    ) / 1000

    self.time_left = max(
        0,
        ROUND_DURATION_SECONDS - elapsed_seconds
    )

    if self.time_left <= 0:
        self.time_left = 0
        self.game_over = True
        return

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_balloon()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    missed_balloons = []

    for balloon in self.balloons:
        balloon.update()

        if balloon.is_past_bottom(HEIGHT):
            missed_balloons.append(balloon)

    for balloon in missed_balloons:
        self.balloons.remove(balloon)
        self.lives -= 1

    if self.lives <= 0:
        self.lives = 0
        self.game_over = True

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(surface, self.balloons)

```

```

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

    renderer.draw_text(
        surface,
        font,
        f"Time: {int(self.time_left)}",
        (10, 70)
    )

    if self.game_over:
        renderer.draw_game_over(
            surface,
            font,
            self.score
        )

```

game/renderer.py

Python

Run

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (200, 230, 245)
COLOR_TEXT = (30, 30, 30)

def draw_scene(surface, balloons):
    surface.fill(COLOR_BG)

    for b in balloons:

```

```

pygame.draw.circle(
    surface,
    b.color,
    (int(b.x), int(b.y)),
    b.radius
)

pygame.draw.line(
    surface,
    (120, 120, 120),
    (b.x, b.y + b.radius),
    (b.x, b.y + b.radius + 12),
    2
)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            surface.get_height() // 2
        )
    )
    surface.blit(surf, rect)

def draw_game_over(surface, font, score):
    overlay = pygame.Surface(
        (surface.get_width(), surface.get_height()),
        pygame.SRCALPHA
    )
    overlay.fill((0, 0, 0, 150))
    surface.blit(overlay, (0, 0))

    game_over_text = font.render(
        "Game Over",
        True,
        (255, 255, 255)
    )

    score_text = font.render(
        f"Final Score: {score}",
        True,
        (255, 255, 255)
    )

    restart_text = font.render(

```

```

        "Press R to Restart",
        True,
        (255, 255, 255)
    )

    center_x = surface.get_width() // 2

    surface.blit(
        game_over_text,
        game_over_text.get_rect(
            center=(center_x, 200)
        )
    )

    surface.blit(
        score_text,
        score_text.get_rect(
            center=(center_x, 240)
        )
    )

    surface.blit(
        restart_text,
        restart_text.get_rect(
            center=(center_x, 280)
        )
    )

```

What changed

- Added a **30-second countdown** using `pygame.time.get_ticks()` , not `time.sleep` .
- The round ends when **time reaches 0 OR lives reach 0**.
- After Game Over, **spawning and balloon clicks stop**, and the **final score** is displayed.
- Pressing **R** completely restarts the round: score → 0, lives → 3, timer → 30, and all balloons are cleared.
- The existing **balloon types, scoring, lives system, and corrected click detection** remain intact.



ChatGPT can make mistakes. Check important info.